Marcos Diaz, Alina Carias, Daniel Machic, Ignacio Méndez, Ariela Mishaan, Diego Soto

Taller 2

1. **Link al repositorio:** https://github.com/ArielaMishaanCohen/Taller_2.git

2. **Capturas de pantalla con terminal mostrando ambiente de python creado**

```
PS C:\Users\Alina Tatjana\OneDrive\Documentos\UVG\10mo SEMESTRE\MLOPS\Taller 2> py -m venv .venv
PS C:\Users\Alina Tatjana\OneDrive\Documentos\UVG\10mo SEMESTRE\MLOPS\Taller 2> .venv\Scripts\activate
(.venv) PS C:\Users\Alina Tatjana\OneDrive\Documentos\UVG\10mo SEMESTRE\MLOPS\Taller 2> Get-Command python
>>

CommandType     Name                                               Version    Source
-----------     ----                                               -------    ------
Application     python.exe                                         3.13.11... C:\Users\Alina Tatjana\OneDrive\Documentos\UVG\10mo SEM...


(.venv) PS C:\Users\Alina Tatjana\OneDrive\Documentos\UVG\10mo SEMESTRE\MLOPS\Taller 2> pip list | Select-Object -First 20
>>
Package Version
------- -------
pip     24.3.1
(.venv) PS C:\Users\Alina Tatjana\OneDrive\Documentos\UVG\10mo SEMESTRE\MLOPS\Taller 2> deactivate
PS C:\Users\Alina Tatjana\OneDrive\Documentos\UVG\10mo SEMESTRE\MLOPS\Taller 2>
```

3. **Capturas de pantalla mostrando la creación del ambiente usando Anaconda/Conda**

```
(base) C:\Users\Alina Tatjana\OneDrive\Documentos\UVG\10mo SEMESTRE\MLOPS\Taller 2\Taller_2\taller2-ambientes>conda env create -f environment.yml
3 channel Terms of Service accepted
Retrieving notices: done
WARNING conda.core.prefix_data:validate_path(345): Environment paths should not contain spaces. Prefix: 'C:\Users\Alina Tatjana\miniconda3\envs\taller2-ml
'
Channels:
 - conda-forge
 - defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done

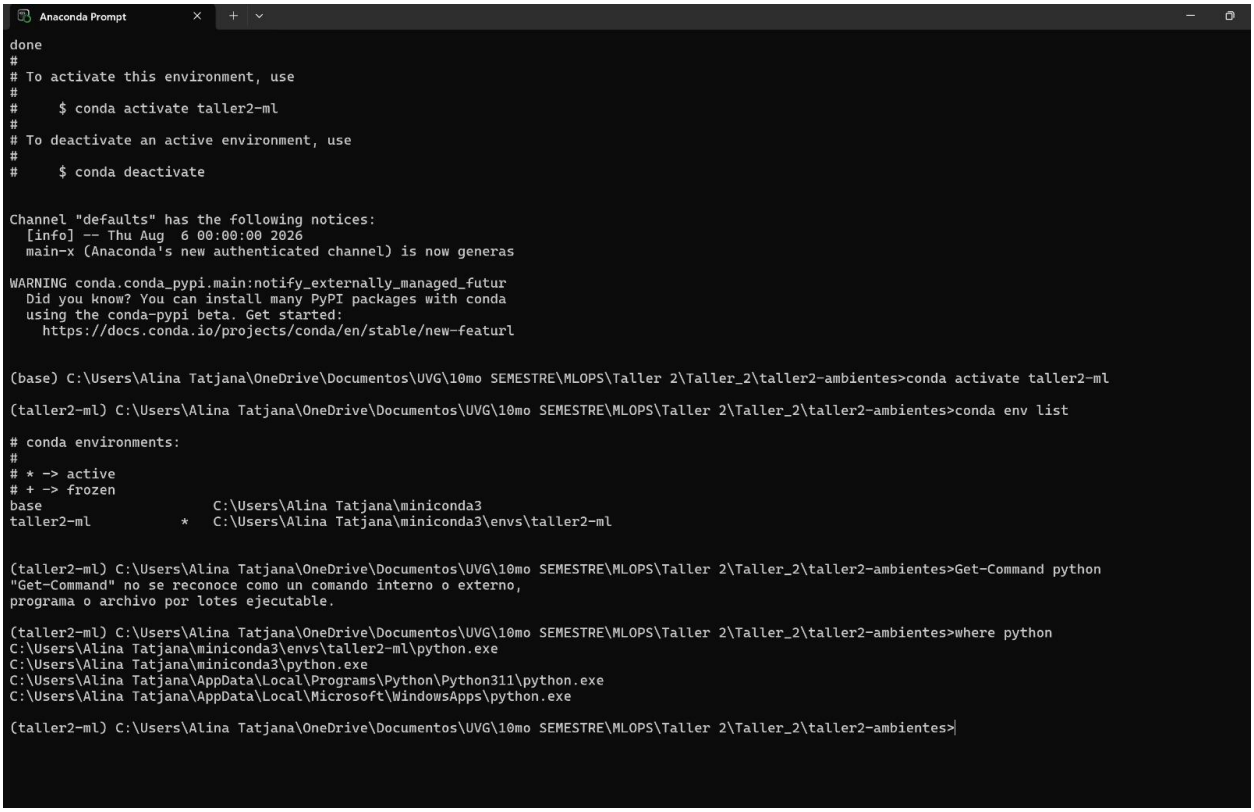
## Package Plan ##

  environment location: C:\Users\Alina Tatjana\miniconda3\envl

  added / updated specs:
    - ipykernel[version='>=6.29,<8.0']
    - joblib[version='>=1.3,<2.0']
    - jupyterlab[version='>=4.0,<5.0']
    - matplotlib[version='>=3.8,<4.0']
    - numpy[version='>=1.26,<3.0']
    - pandas[version='>=2.1,<3.0']
    - pip
    - pytest[version='>=8.0,<9.0']
    - python=3.12
    - scikit-learn[version='>=1.4,<2.0']
    - seaborn[version='>=0.13,<1.0']


The following packages will be downloaded:

    package                    |            build
    ---------------------------|-----------------
    _python_abi3_support-1.0   |       hd8ed1ab_3           8e
    anyio-4.15.1               |     pyh5ded981_1         171e
    argon2-cffi-25.1.0         |     pyhd8ed1ab_0          18e
    argon2-cffi-bindings-26.1.0|  py312he06e257_2          33e
    arrow-1.4.0                |     pyhcf101f3_0         111e
    asttokens-3.0.2            |     pyhd8ed1ab_0          34e
    async-lru-2.3.0            |     pyhcf101f3_0          22e
    attrs-26.1.0               |     pyhcf101f3_0          63e
    babel-2.18.0               |     pyhcf101f3_1         7.3e
    backports.zstd-1.7.0       |  py312h06d0912_1         234e
    beautifulsoup4-4.15.0      |     pyha770c72_0          91e
    bleach-6.4.0               |     pyhcf101f3_0         139e
```



4. **Conclusiones del ejercicio pensando posibles usos de los ambientes virtuales en un proyecto de Machine Learning.**

   El uso de ambientes virtuales es fundamental en proyectos de Machine Learning porque permite aislar las dependencias de cada proyecto y evitar conflictos entre versiones de librerías como pandas, NumPy o scikit-learn. Durante el ejercicio se observó que tanto venv como Conda permiten crear espacios de trabajo independientes, pero Conda además facilita la gestión de paquetes y de versiones de Python dentro del mismo entorno.

   Esto es especialmente útil en Machine Learning, donde los proyectos suelen depender de muchas librerías y, en algunos casos, de componentes que no son exclusivamente de Python. También resulta importante mantener un archivo que describa las dependencias del proyecto, ya sea requirements.txt, environment.yml o un archivo de bloqueo, porque esto permite que otra persona pueda reproducir el entorno y ejecutar el proyecto con las mismas versiones. En un trabajo colaborativo o al momento de desplegar un modelo, esta reproducibilidad reduce errores y facilita el mantenimiento. En conclusión, los ambientes virtuales no solo ayudan a mantener ordenado el equipo de desarrollo, sino que forman parte de una práctica necesaria para construir proyectos de Machine Learning más confiables, reproducibles y fáciles de compartir.

5. Tabla comparativa: Conda vs. UV

Aspecto	Conda	UV
Enfoque principal	Gestión de ambientes y paquetes, especialmente popular en ciencia de datos y computación científica.	Gestor moderno de proyectos, paquetes y ambientes de Python, enfocado en velocidad y compatibilidad con el ecosistema Python.
Creación de ambientes	conda create -n nombre python=3.x	uv venv o creación automática del ambiente al trabajar con un proyecto.
Gestión de dependencias	Instala paquetes desde canales de Conda y también puede combinarse con pip.	Resuelve e instala dependencias de Python con comandos como uv add y uv sync.
Archivos para reproducibilidad	environment.yml puede guardar dependencias, canales y versión de Python.	pyproject.toml define el proyecto y uv.lock registra versiones exactas para reproducir el entorno.
Paquetes no Python	Sí. Puede administrar librerías y dependencias externas a Python.	No es su objetivo principal; se concentra en paquetes y herramientas del ecosistema Python.
Velocidad	Puede ser más lento al resolver e instalar ambientes grandes.	Su implementación en Rust está diseñada para resolver e instalar dependencias con mucha rapidez.
Manejo de Python	Permite crear ambientes con versiones específicas de Python.	Además de seleccionar versiones, puede instalar y administrar versiones de Python.
Uso en Machine Learning	Muy útil cuando el proyecto necesita paquetes científicos, binarios o dependencias del sistema.	Muy útil para proyectos Python que buscan instalaciones rápidas, reproducibilidad y un flujo de trabajo más simple.
Ventaja principal	Amplio manejo de paquetes científicos y dependencias que no son únicamente de Python.	Integra en una sola herramienta tareas que normalmente requieren pip, venv y otras utilidades.

Fuente de apoyo: DataCamp, "Python UV: The Ultimate Guide to the Fastest Python Package Manager".

6. Tabla comparativa: Conda vs. Poetry

Aspecto	Conda	Poetry
Enfoque principal	Gestión de ambientes y paquetes con fuerte presencia en ciencia de datos.	Gestión integral de proyectos Python, dependencias, ambientes virtuales y empaquetado.
Creación de ambientes	conda create -n nombre python=3.x	Poetry crea y administra automáticamente un ambiente virtual asociado al proyecto.
Gestión de dependencias	Usa conda install y repositorios/canales de Conda; también puede utilizar pip.	Usa poetry add, poetry remove y poetry install con resolución automática de dependencias.
Archivo de configuración	environment.yml para describir el ambiente y sus dependencias.	pyproject.toml centraliza metadatos y dependencias del proyecto.

Aspecto	Conda	Poetry
Archivo de bloqueo	Puede definir versiones en environment.yml, pero no trabaja igual que un lockfile dedicado.	poetry.lock guarda las versiones exactas de dependencias y subdependencias para reproducibilidad.
Paquetes no Python	Sí, puede manejar bibliotecas y dependencias externas a Python.	No; está orientado al ecosistema y empaquetado de Python.
Publicación de paquetes	No está centrado en publicar paquetes Python a PyPI.	Incluye comandos para construir y publicar paquetes en PyPI.
Organización del proyecto	El ambiente puede existir de forma independiente a la estructura del proyecto.	Tiene un enfoque más estructurado y orientado al ciclo de vida completo de un proyecto Python.
Uso en Machine Learning	Conveniente cuando hay dependencias científicas complejas o componentes externos a Python.	Conveniente cuando se quiere controlar versiones, colaborar y mantener un proyecto Python reproducible a largo plazo.
Ventaja principal	Flexibilidad para administrar Python y dependencias de sistema en un mismo ambiente.	Unifica dependencias, entorno virtual, configuración y empaquetado del proyecto.

Fuente de apoyo: DataCamp, "Python Poetry: Modern And Efficient Python Environment And Dependency Management".